



RISC-V 기반 AMBA Peripheral 설계

이윤지



목차

01 개요



02 Multi Cycle



03 AMBA



04 UART



05 C & 동작 영상



06 Trouble shooting



07 고찰



08 Reference

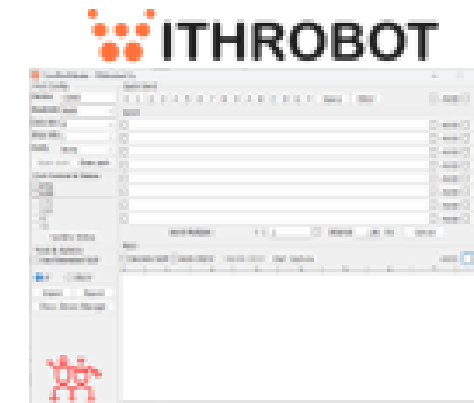


개요

● 프로젝트 목표

- RISC-V RV32I 기반 Muti-Cycle CPU CORE 설계
- AMBA APB 기반 Peripheral 연결
- SystemVerilog를 이용한 Peripheral 검증
- C언어를 통한 Application Code 작성 및 동작 구현

● 구현환경



ComportMaster



AMBA



RISC-V



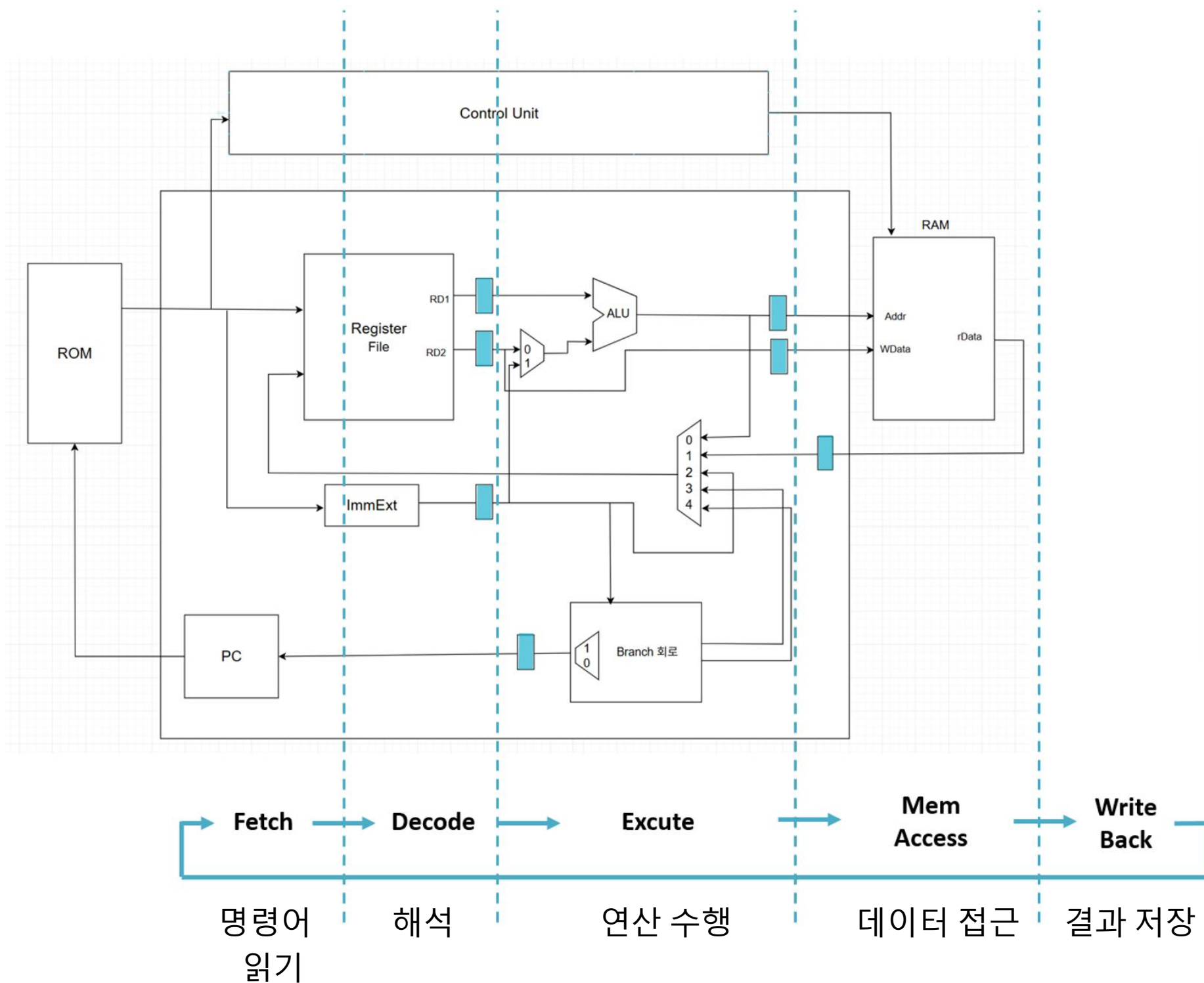
SystemVerilog



CHAPTER 02



Multi Cycle



구분	Single Cycle	Multi Cycle
명령어 수행	한 클럭에 모든 단계 수행	여러 클럭에 수행
속도	가장 느린 명령어에 맞춰 클럭 길이가 김	짧은 클럭으로 명령어를 나눠 수행
장점	구조 단순	빠른 클럭, 효율적
단점	비효율적	설계 복잡

Multi Cycle

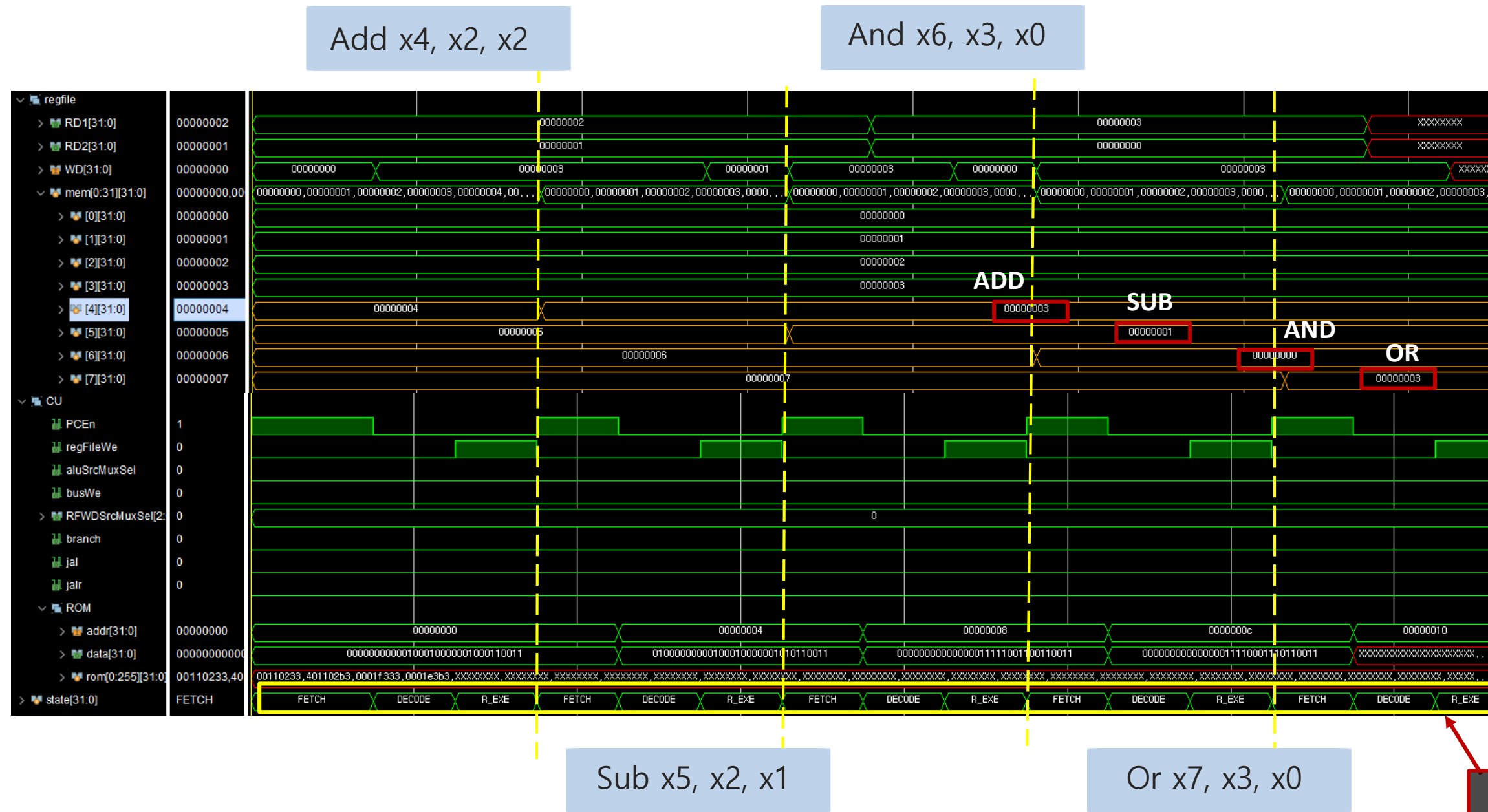
- 기존 Single Cycle에 레지스터 삽입
- 하나의 명령어를 여러 클럭 주기에 걸쳐 처리
- 명령어 Fetch → Decode → Execute → Memory → Writeback 단계를 각각 다른 사이클에서 수행
- 동일한 ALU나 메모리를 여러 단계에서 재사용 가능



CHAPTER 02



Multi Cycle



R-Type

- 역할 : 레지스터 간 연산 수행
- 수행 단계 : Fetch -> Decode -> Excute
- ALU만 사용하고, 결과를 바로 레지스터에 저장해 불필요한 Memory 단계는 생략
- Excute 시점 : 지정한 레지스터 주소(rd)에 연산 결과를 저장

필요한 단계까지만 수행하여 클럭 낭비를 줄인다.

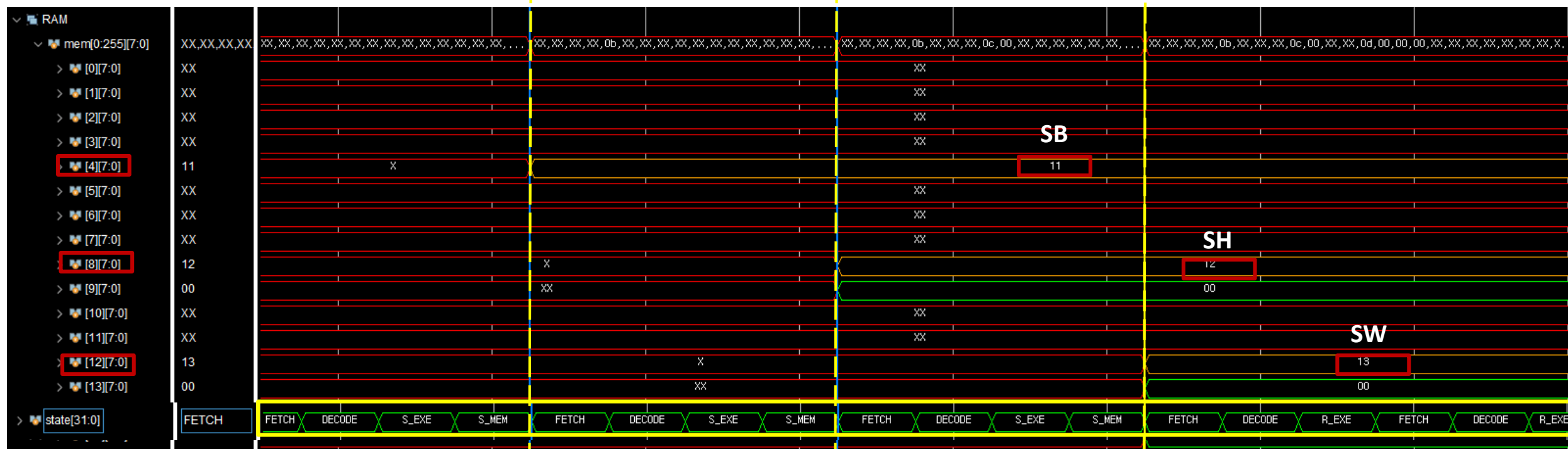


CHAPTER 02

Multi Cycle

SB x11, x4(0)

SW x13, x12(0)



SH x12, x8(0)

S-Type

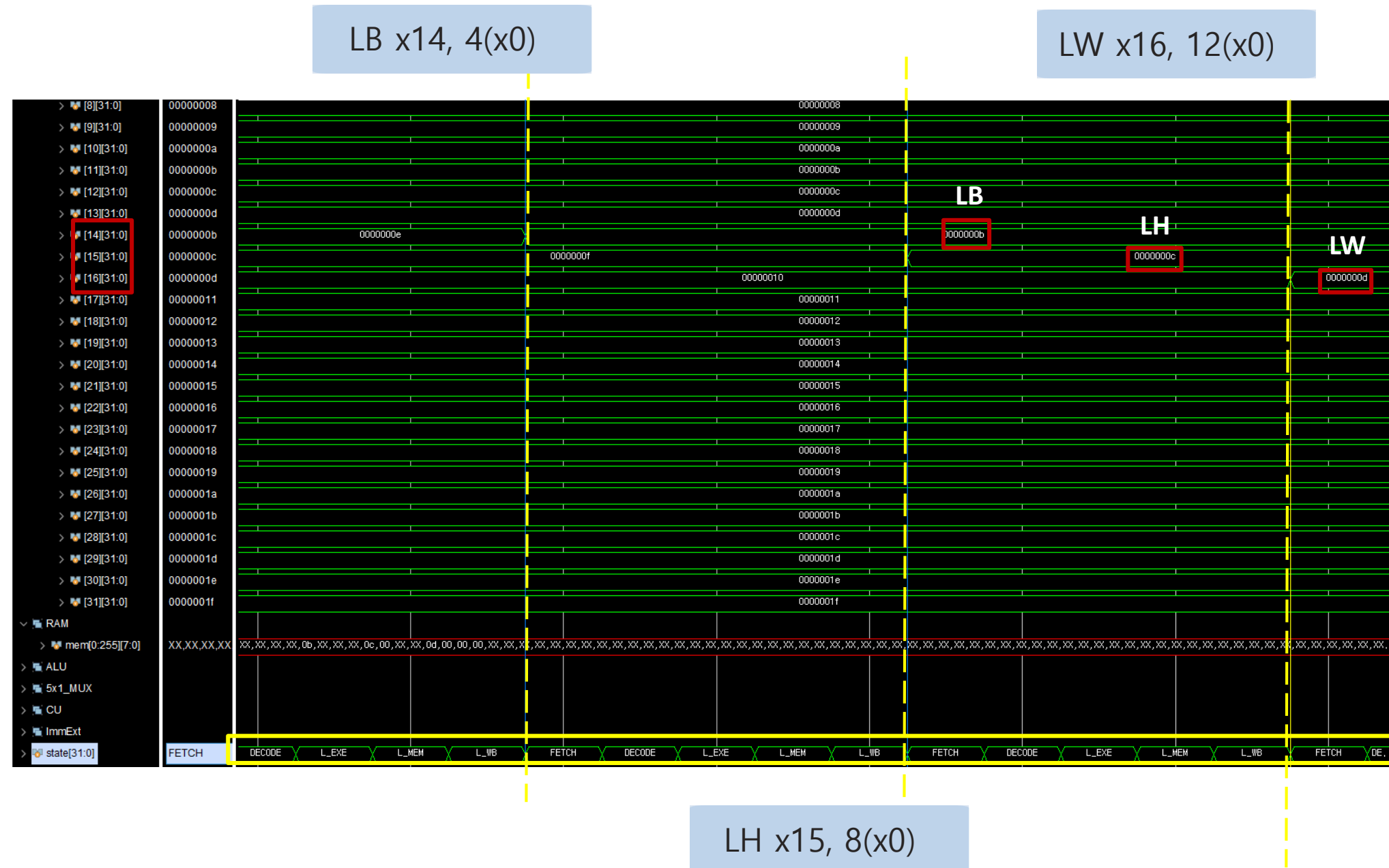
- 역할 : 레지스터 값을 메모리에 저장
- 수행 단계 : Fetch -> Decode
-> Excute -> Memory Access
- 레지스터의 데이터를 메모리에 저장하며, Writeback 단계는 생략
- Memory Access 시점 : 지정한 레지스터 주소(rs2) 에 연산 결과를 저장



CHAPTER 02



Multi Cycle



L-Type

- 역할 : 메모리 값을 레지스터로 불러옴
- 수행 단계 : Fetch -> Decode
-> Excute -> Memory Access
-> Write Back
- 메모리에서 읽은 데이터를 레지스터에 저장하기 때문에 모든 단계를 거치며, ALU는 주소 계산에 사용
- Write Back 시점 : 지정한 레지스터 (rd) 에 메모리에서 읽어온 데이터를 저장

“즉 Single-Cycle 과 달리 명령어 별로 단계를 유연하게 분리해 효율적으로 사용 가능”



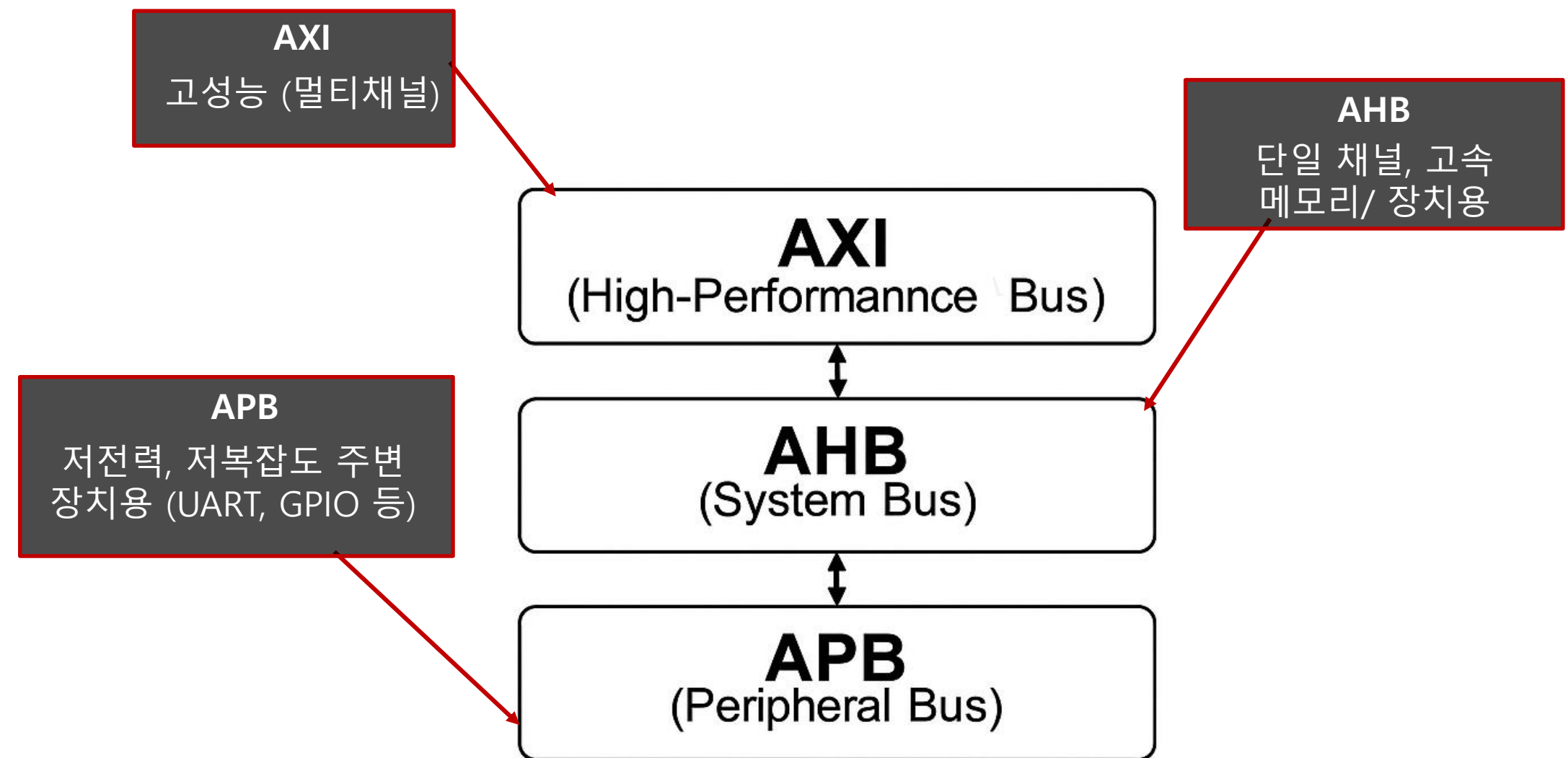
AMBA

AMBA?

- AMBA (Advanced Microcontroller Bus Architecture)
 - > ARM에서 제안한 SoC 내부 버스 표준 구조
 - > AXI, AHB, APB 계층적 구조

사용 이유?

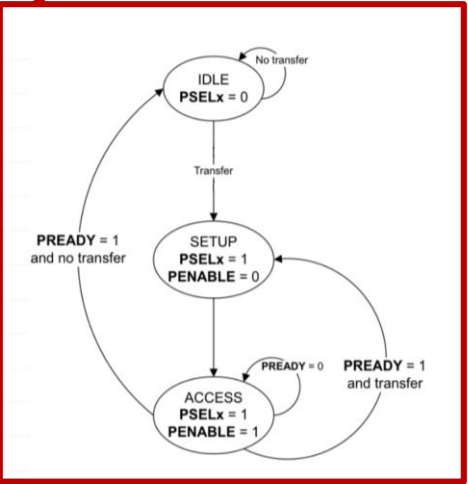
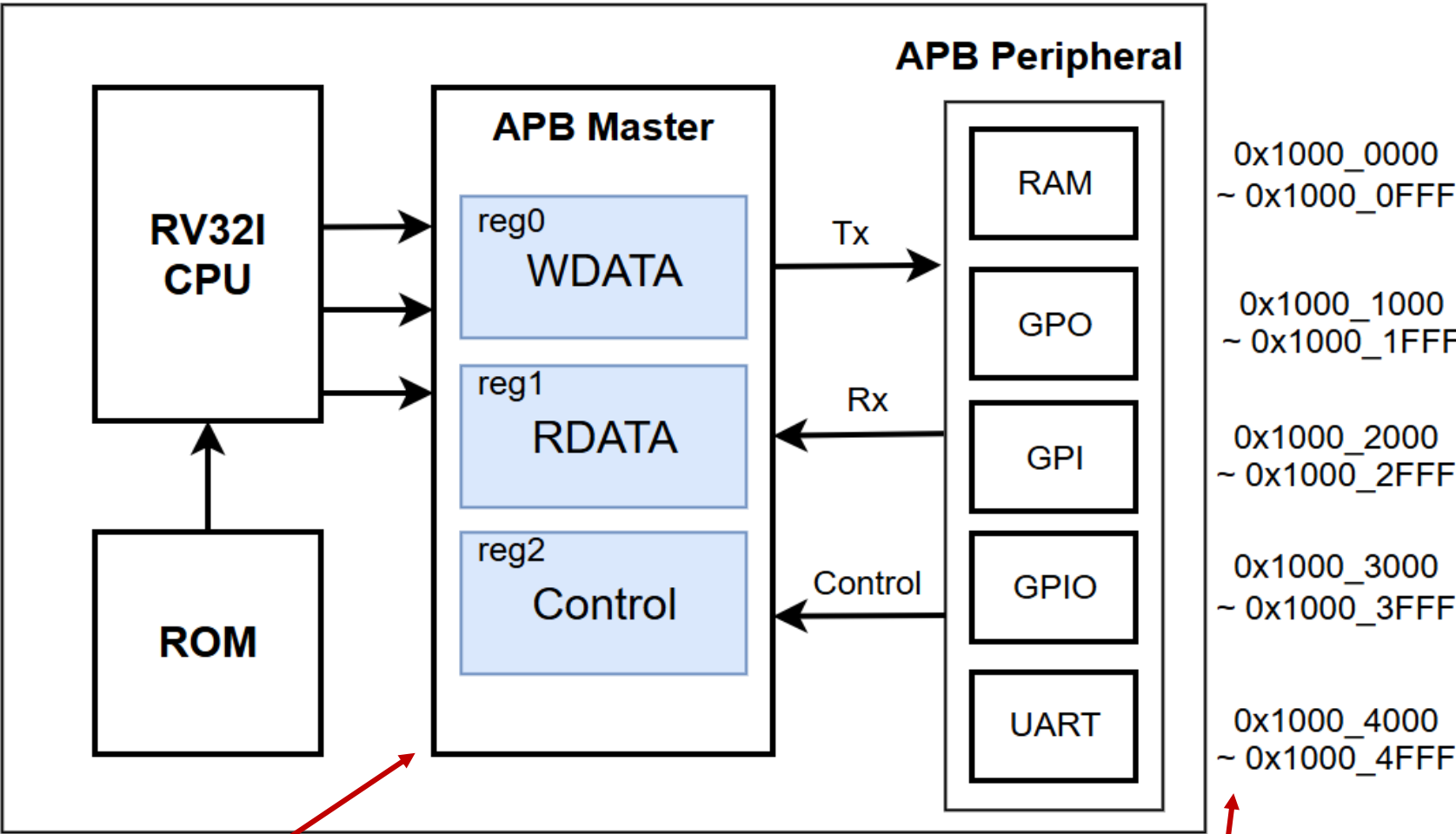
- 여러 IP(Core, Memory, Peripheral)를 하나의 규격으로 연결
- 계층적 구조 (AXI / AHB / APB)
 - > 성능, 복잡도, 전력 효율성에 맞는 선택 가능
- 저전력, 저비용 설계 용이
 - > 특히 APB는 주변 장치 제어에 최적





AMBA

MCU



APB Master의 FSM
3단계 전송 절차(IDLE → SETUP → ACCESS)

Memory Map
각 Peripheral은 독립된 주소 공간을 할당받아, CPU는 해당 주소를 통해 직접 접근이 가능

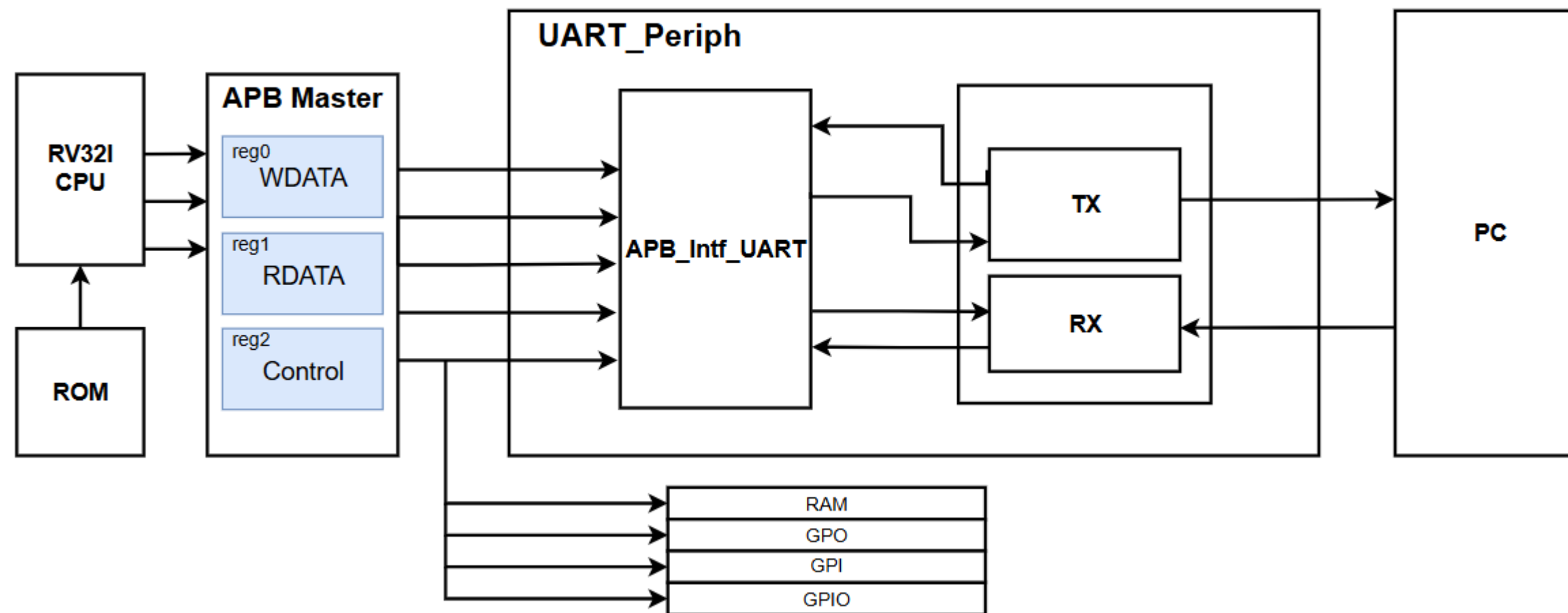
APB

- 역할 : CPU가 제어 레지스터를 통해 주변장치에 접근하는 구조
- CPU는 APB Master를 통해 레지스터 사용
- APB Master는 3단계를 거쳐 Peripheral과 데이터 통신 수행
- Reg0 (WDATA) : 데이터 전송
- Reg1 (RDATA) : 수신 데이터 읽음
- Reg2 (Control) : 신호 저장

CHAPTER 04



UART Peripheral



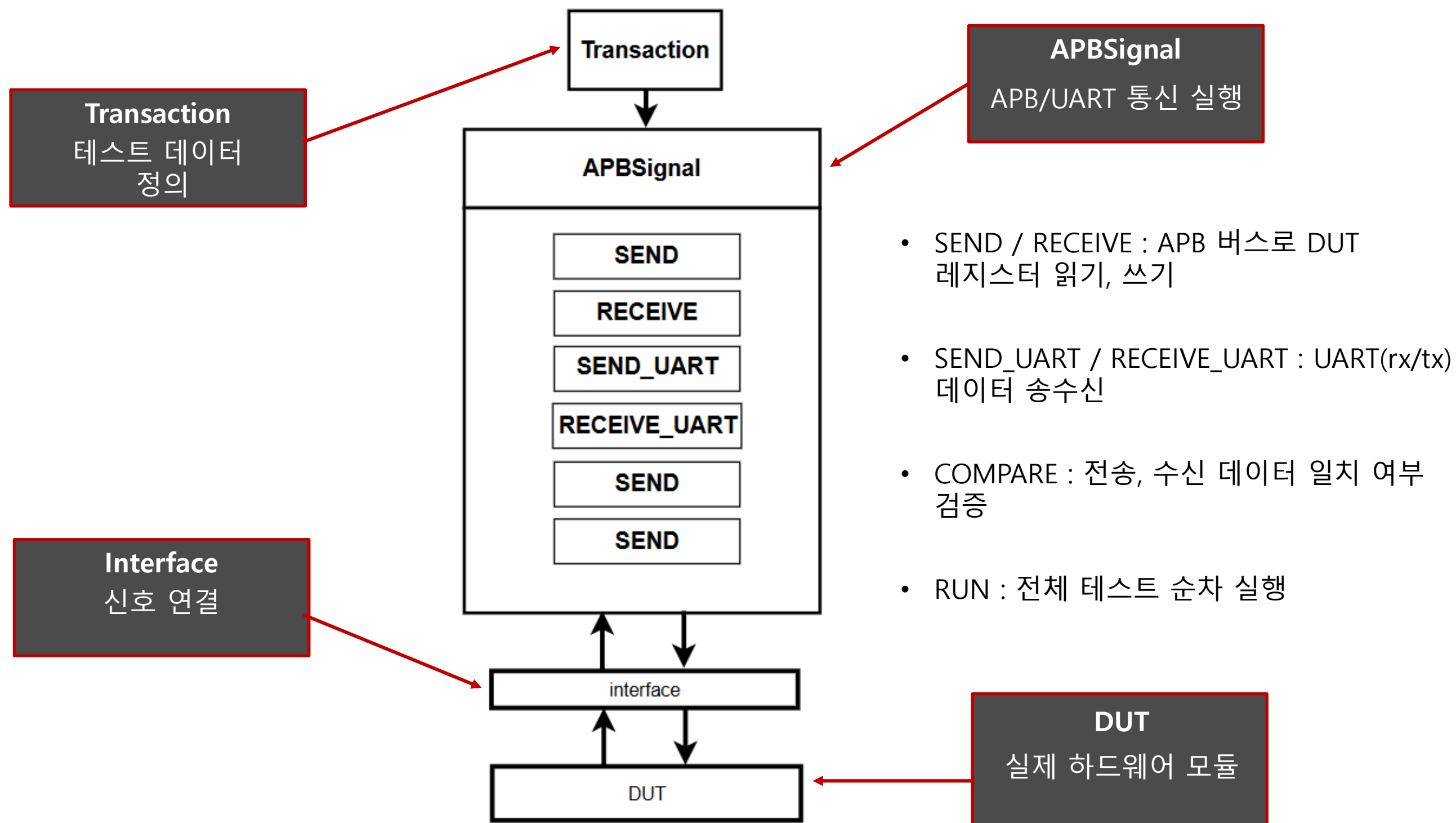
UART

- APB Master : UART 인터페이스를 통해 Rx, TX와 데이터 교환
- CPU : 레지스터로 UART 제어
-> APB 레지스터를 통해 UART 동작 제어
- Tx : CPU 데이터 -> PC 로 전송
- Rx : PC 데이터 -> CPU로 전달



CHAPTER 04

UART 검증



Verification

- UART Peripheral의 TX/RX 동작
- TX: CPU가 APB를 통해 전송한 데이터가 UART TX 경로를 통해 정상 전달
- RX: UART가 수신한 데이터가 내부 레지스터에 올바르게 저장

```

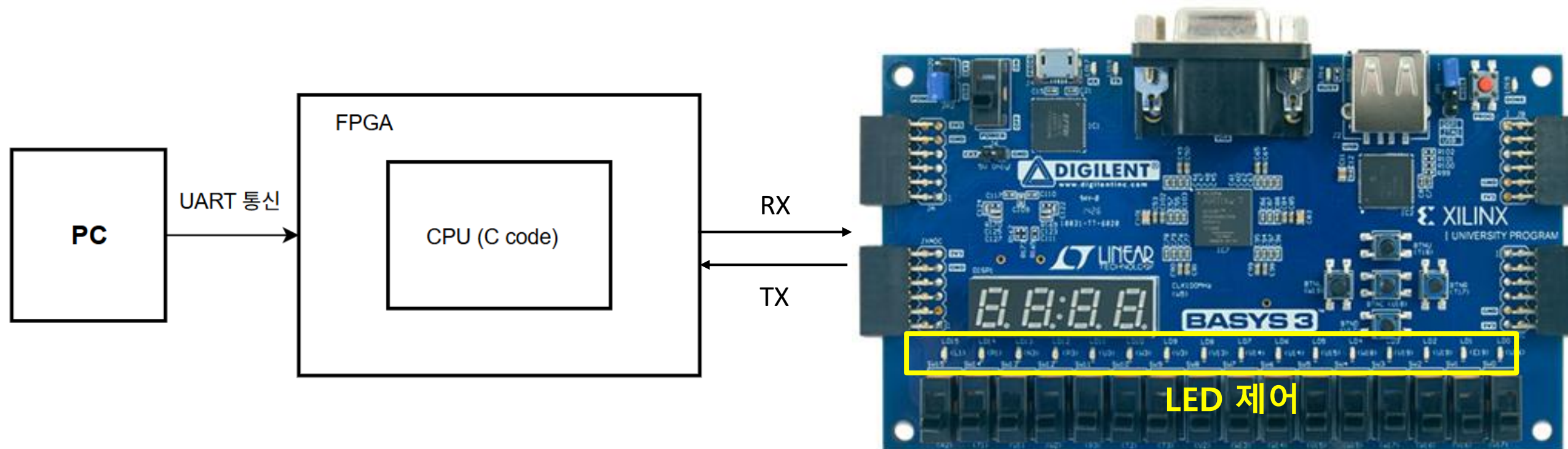
===== Test #255 =====
[510043605000][APB_WRITE] addr=10004000 wdata=00000040 rdata=bbcd4008 send=2f rcv=3f
[511043605000][UART_TX_RECV] addr=10004000 wdata=00000040 rdata=bbcd4008 send=2f rcv=40
[511043605000][UART_RX_SEND] addr=10004000 wdata=00000040 rdata=bbcd4008 send=2f rcv=40
[512043655000][APB_READ] addr=10004004 wdata=00000040 rdata=bbcd4004 send=2f rcv=40
[512043715000][APB_READ] addr=10004008 wdata=00000040 rdata=bbcd4008 send=2f rcv=40
TX PASS! RX PASS!
=== ALL TEST PASS ===
===== Simulation Complete =====
  
```

All Test Pass

CHAPTER 05



C application



C application

- 역할 : CPU가 실행되며 APB를 통해 FPGA 내부의 주변 장치를 제어
- PC → UART 통신 → CPU(C 코드 실행) → APB → 주변 장치
- CPU는 APB 주소 매핑을 통해 하드웨어 레지스터에 직접 접근하며, 명령에 따라 LED 등 주변 장치의 상태를 제어



CHAPTER 05



C application

```
// 'A' 입력 시: LED ON, 'O' 한 번만 출력
if (c == 'A') {
    blink_mode = 0;
    GPO_ODR = 0xFFFF;

    if (sent_flag_A == 0) {
        delay(20000);
        UART_send_char('O'); // 'O' 전송
        delay(20000);
        sent_flag_A = 1;
        sent_flag_B = 0;
        sent_flag_C = 0;
    }
}

// 'B' 입력 시: LED 깜빡거림, 'B' 한 번만 출력
else if (c == 'B') {
    blink_mode = 1;

    if (sent_flag_B == 0) {
        delay(20000);
        UART_send_char('B'); // 'B' 전송
        delay(20000);
        sent_flag_B = 1;
        sent_flag_A = 0;
        sent_flag_C = 0;
    }
}
```

```
// 'C' 입력 시: LED OFF, 'X' 한 번만 출력
else if (c == 'C') {
    blink_mode = 0;
    GPO_ODR = 0x0000;

    if (sent_flag_C == 0) {
        delay(20000);
        UART_send_char('X'); // 'X' 전송
        delay(20000);
        sent_flag_C = 1;
        sent_flag_A = 0;
        sent_flag_B = 0;
    }
}

// 깜빡임 모드일 때 LED
if (blink_mode) {
    GPO_ODR = 0xFFFF;
    delay(1000000);
    GPO_ODR = 0x0000;
    delay(1000000);
}
```

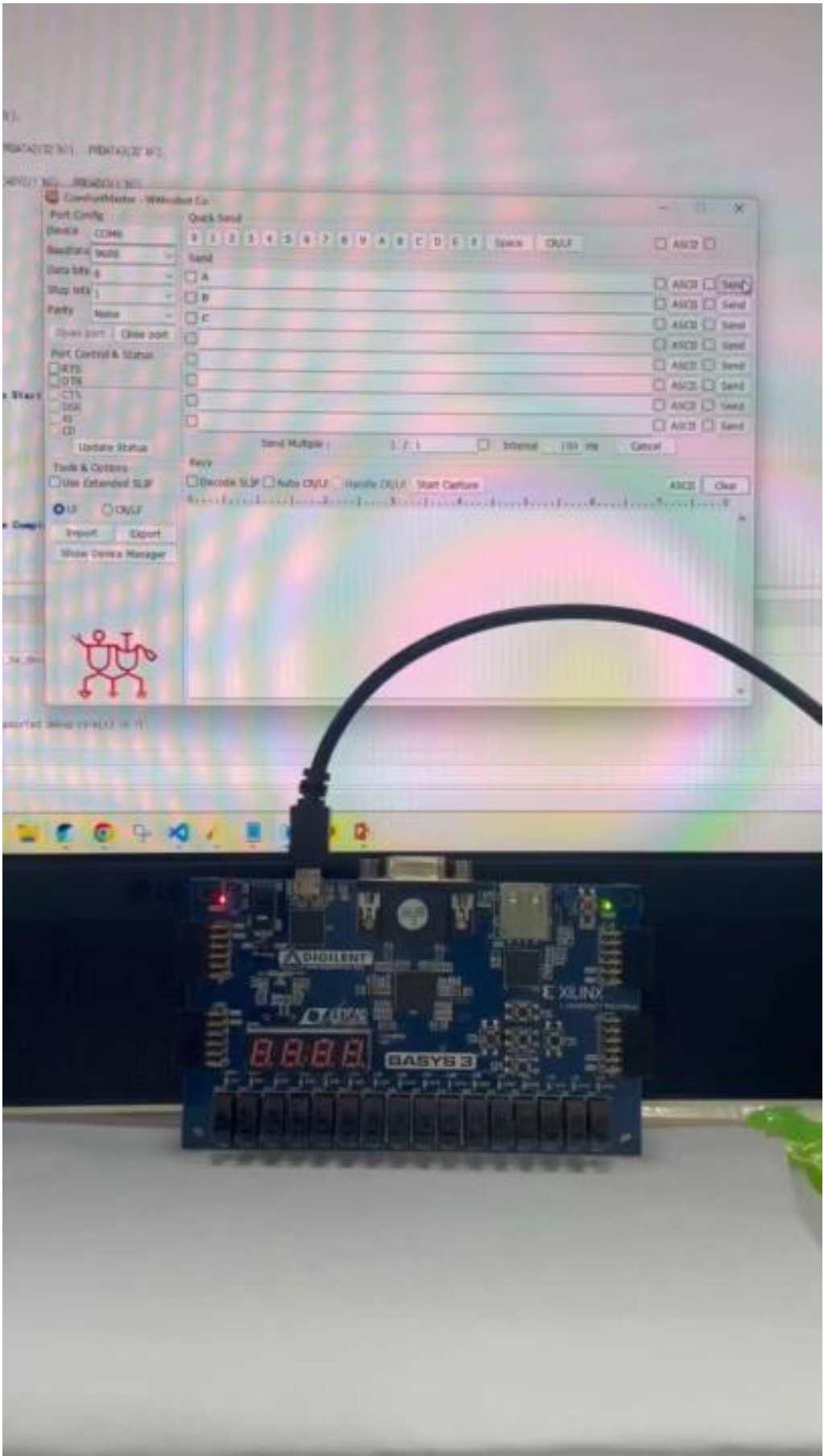
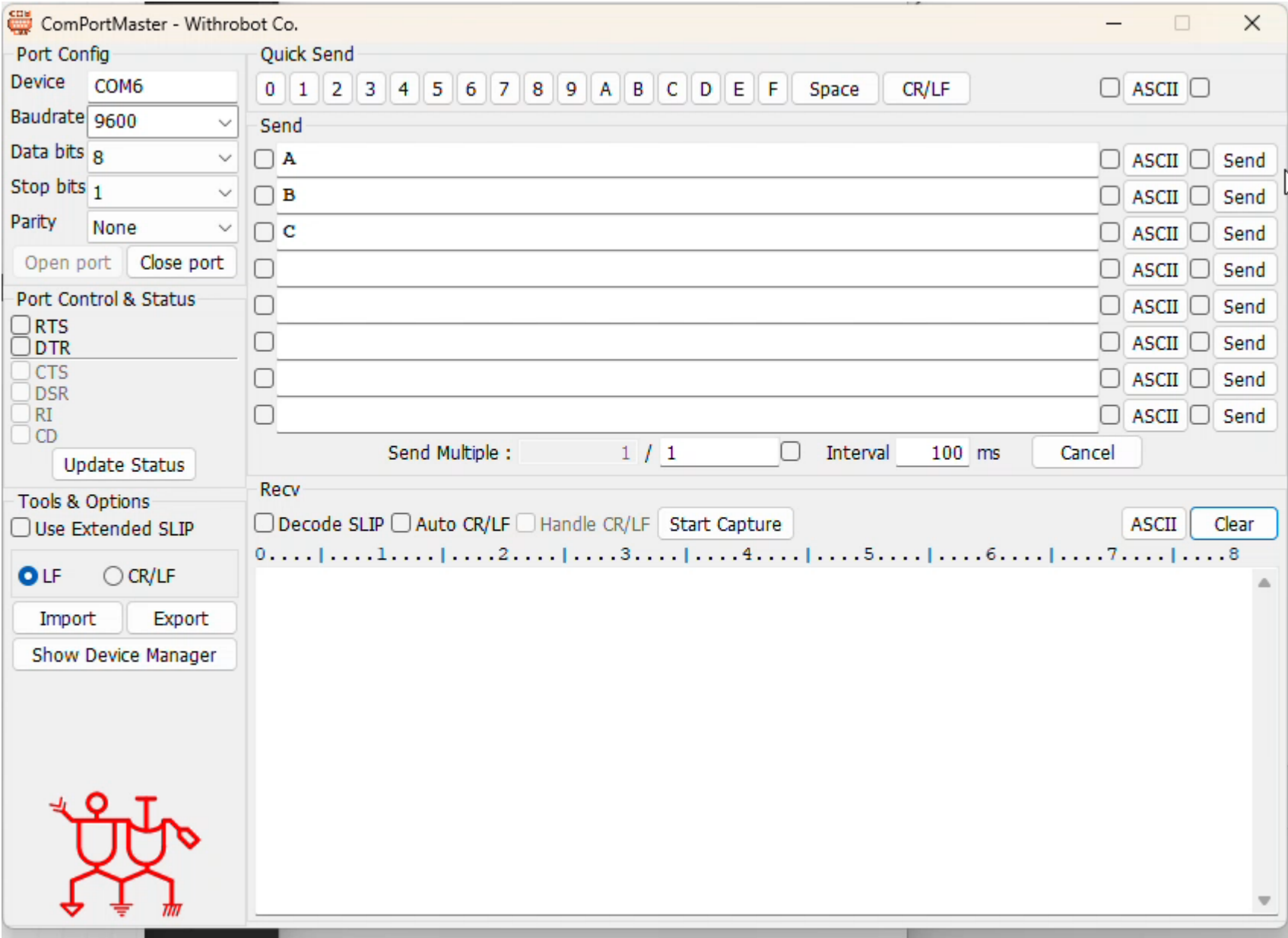
C application

- 'A' → LED ON
- 'B' → LED BLINK
- 'C' → LED OFF
- 각각의 동작 후 UART_TX를 통해 응답 문자 전송 ('O', 'B', 'X')



CHAPTER 05 🔍

동작 영상





Trouble Shooting

문제 발생

```
uart_tx U_UART_TX(
    .clk(PCLK),
    .rst(PRESET),
    .start_trigger(tx_start | rx_done),
    .tx_data(tx_data | rx_data),
    .b_tick(w_b_tick),
    .tx(tx),
    .tx_busy(tx_busy)
);
```

- Uart_tx 인스턴스 시 start_trigger, tx_data를 OR 연산자로 신호를 묶음
- Rx_done 때 tx가 강제로 시작돼 데이터 중복 전송
- 문자 'O' (0x4F, 0100_1111)
- 문자 'X' (0x58, 0101_1000) 이 동시에 활성화되어
- 결과가 0101_1111 (0X5F)로 OR 연산되어 데이터 전송

문제 해결

```
uart_tx U_UART_TX(
    .clk(PCLK),
    .rst(PRESET),
    .start_trigger(tx_start),
    .tx_data(tx_data),
    .b_tick(w_b_tick),
    .tx(tx),
    .tx_busy(tx_busy)
);
```

- Tx_start 신호로만 tx 동작 제어
- Rx 완료 신호 (rx_done)는 독립적으로 처리
- 즉, 한쪽 통신이 끝난 후에만 다른 쪽 동작을 하도록 분리



고찰

이번 프로젝트를 진행하면서 단순히 UART 통신을 구현하는 것을 넘어, AMBA-APB 구조의 필요성과 CPU가 주변장치를 제어하는 원리를 보다 명확하게 이해할 수 있었다.

C 코드와 Verilog가 버스를 통해 실제로 데이터를 주고받는 과정을 직접 확인하면서, 데이터가 어떤 경로를 따라 이동하고, 어떤 신호에 의해 제어되는지를 구체적으로 파악했다.

특히 발표를 준비하며 구조를 다시 정리하는 과정에서 APB Master와 Peripheral 간의 데이터 흐름을 명확히 정리할 수 있었다.

이를 통해 단순히 모듈 단위가 아닌 시스템 전체의 데이터 전송 구조를 이해하는 관점을 가지는 기회였던 것 같다.

마지막으로, 이번 결과를 기반으로 FND(7-Segment) 디스플레이를 연동해 데이터를 시각화하거나,

TX 데이터에 문장 형태의 정보를 전송하는 방식으로 기능을 확장해보고 싶다는 생각을 했다.



Reference

그림

ARM Ltd., 『**AMBA AHB 프로토콜 명세서 (IHI 0033C)**』, 2021.

ARM 공식 기술문서, <https://developer.arm.com/documentation/ihi0033/latest>



Thank You